

Altar Server Rota System

Technical Design Document

Node.js + TypeScript + PostgreSQL

Draft

1. Purpose

This document defines the recommended technical architecture for the Altar Server Rota System. It supports the functional requirements in `scheduler-requirements.typ` and codifies the selected implementation stack:

- Node.js runtime
- TypeScript application code
- Next.js web application with React
- PostgreSQL database
- Docker Compose development and deployment baseline
- TypeScript scheduling engine

The existing Lua scheduler is treated as a prototype for scheduling concepts and test data. The production application should implement the scheduling engine in TypeScript so that it shares the same domain model, validation rules, and database access layer as the web application.

2. Architectural Decision

ADR-001. The system shall be implemented primarily in TypeScript running on Node.js.

ADR-002. The web application shall use Next.js with React and the Next.js App Router.

ADR-003. PostgreSQL shall be the system of record.

ADR-004. The scheduling engine shall be implemented as a TypeScript domain module, not as database stored procedures.

ADR-005. The application shall use server-side authorization checks for every operation that creates, changes, or deletes rota data.

ADR-006. The application shall expose a responsive web interface for desktop, tablet, and mobile browsers.

ADR-007. The baseline runtime architecture shall use Docker Compose with separate application and database containers.

2.1. Rationale

Next.js gives the project a single TypeScript codebase for the user interface, server-side rendering, form handling, API routes, and deployment. PostgreSQL is preferred over a NoSQL database because the core domain is relational: adult accounts manage server profiles, server profiles hold qualifications, services contain positions, and positions receive assignments and volunteer requests.

PostgreSQL constraints should enforce critical invariants where practical, while application services enforce workflow rules and domain-specific authorization.

3. System Context

The system will be used by:

- Adult users with login accounts
- Managing adults who manage minor server profiles
- Schedulers with rota-management privileges

The system will provide:

- Calendar and rota views
- Account and server profile management
- Service and position management
- Volunteer request submission
- Scheduler approval workflows
- Generated rota proposals
- Assignment and decline tracking
- Audit/change records

4. Technology Stack

TST-001. The frontend shall be built with React components in a Next.js application.

TST-002. The backend shall run on Node.js and be written in TypeScript.

TST-003. The database shall be PostgreSQL.

TST-004. The application shall use a TypeScript-safe database access layer.

TST-005. The project shall use Prisma for database access and migrations.

TST-006. The project shall use database migrations for all schema changes.

TST-007. The project shall use automated tests for domain logic, scheduling rules, and permission checks.

Recommended default choices:

- Next.js App Router
- React Server Components where appropriate
- TypeScript strict mode
- PostgreSQL
- Prisma for schema migrations and database access
- Auth.js for authentication
- Zod or similar schema validation for form/API input
- Vitest for unit tests
- Playwright for browser workflow tests

TST-008. The project shall use Prisma as the primary database access layer and migration tool.

TST-009. The project shall use Auth.js for authentication.

5. Logical Architecture

The application should be organized into the following modules:

accounts:: Adult login accounts, scheduler flag, authentication identity, and profile ownership.

server-profiles:: Adult and minor schedulable people, including the relationship between a minor and the Managing Adult.

roles:: Liturgical roles and role qualifications.

services:: Liturgical services such as Mass, Vespers, Benediction, or other parish liturgies.

positions:: Required slots within a service.

rotas:: Calendar-facing collections of services and assignments over a date range.

assignments:: Server profile placements into service positions.

volunteer-requests:: Requests submitted by adult accounts for eligible server profiles.

scheduler-engine:: Generation of proposed assignments from services, positions, qualifications, availability, and existing assignments.

audit-log:: Change records for user actions and scheduler actions.

6. Deployment Architecture

The initial deployment shall use a simple two-container Docker Compose architecture:

1. `app`: Next.js web application running on Node.js
2. `db`: PostgreSQL database with persistent storage

The application and database shall be separate containers because the application is disposable and the database is stateful. The database container shall use a named Docker volume for PostgreSQL data.

The first implementation may run scheduling generation synchronously in the application container if generation is fast enough. If generation becomes slow or needs retries, the system may add a separate worker container later.

DEP-001. The project shall include a Dockerfile for the application container.

DEP-002. The project shall include a Docker Compose file defining `app` and `db` services.

DEP-003. The `app` service shall run the Next.js application.

DEP-004. The `db` service shall run PostgreSQL.

DEP-005. The `db` service shall store PostgreSQL data in a named Docker volume.

DEP-006. The `app` service shall connect to PostgreSQL through Docker Compose service networking.

DEP-007. Database connection settings shall be provided through environment variables.

DEP-008. The project shall include an example environment file documenting required configuration values.

DEP-009. The application container shall not store persistent application data on its writable filesystem.

DEP-010. A separate worker container may be added later using the same application image with a different start command.

6.1. Optional Cloud Deployment

Docker Compose is the baseline for local development and may be sufficient for a small self-hosted deployment. Cloud deployment is optional and should not change the application architecture.

For cloud production deployments, the recommended shape is:

1. containerized `app` service
2. managed PostgreSQL database
3. managed configuration and secrets
4. provider-managed logs and backups

In cloud production, PostgreSQL should usually be provided by a managed database service rather than by a PostgreSQL container. The application should continue to connect through the same environment-based database configuration used locally.

AWS reference option:

- app runtime: AWS App Runner or ECS Fargate
- database: Amazon RDS for PostgreSQL
- secrets/configuration: AWS Secrets Manager or Systems Manager Parameter Store
- logs: Amazon CloudWatch
- TLS/domain: managed by App Runner or by an Application Load Balancer with ACM

Azure reference option:

- app runtime: Azure Container Apps or Azure App Service for Containers
- database: Azure Database for PostgreSQL Flexible Server
- secrets/configuration: Azure Key Vault or managed application settings
- logs: Azure Monitor and Log Analytics
- TLS/domain: managed by the selected Azure app hosting service, Front Door, or Application Gateway

CLD-001. Cloud deployment shall be optional.

CLD-002. Cloud deployment shall not require a different application architecture.

CLD-003. The same application container image should be usable in Docker Compose, AWS, Azure, or another container runtime.

CLD-004. For cloud production deployment, the application should use managed PostgreSQL rather than running PostgreSQL in an application-managed container.

CLD-005. Cloud deployments shall provide database connection settings through environment variables or the provider's managed configuration mechanism.

CLD-006. Cloud deployments should use the provider's managed secrets service for sensitive values.

CLD-007. Cloud deployments should use provider-managed logging.

CLD-008. Cloud deployments should rely on managed database backup and restore features.

7. Data Model

The following tables are recommended as the first relational model. Names are descriptive rather than final.

7.1. user_accounts

- id
- login_name
- password_hash
- first_name

- middle_name
- last_name
- email
- phone_primary
- phone_secondary
- is_scheduler
- created_at
- updated_at
- disabled_at

Constraints:

- `login_name` unique
- `email` unique, if email is required for login or password reset
- all User Accounts represent adults
- authentication identities are managed through Auth.js

7.2. auth_accounts_and_sessions

Auth.js shall provide the authentication account/session tables or equivalent persistence model required by the selected Auth.js adapter.

The application domain shall not treat external identity provider accounts as Server Profiles. External authentication proves the adult user's login identity; application tables still control Scheduler privileges, managed Minor Server Profiles, Role Qualifications, Assignments, and Volunteer Requests.

7.3. server_profiles

- id
- first_name
- middle_name
- last_name
- profile_type: adult or minor
- linked_user_account_id, nullable
- managing_user_account_id, nullable
- is_active
- created_at
- updated_at

Constraints:

- adult profiles may have a linked User Account
- minor profiles shall have exactly one Managing Adult
- minor profiles shall not have a linked login account
- adult profiles shall not have a Managing Adult unless a future requirement explicitly allows assisted management

7.4. roles

- id
- code
- name

- description
- display_order
- is_active

Constraints:

- `code` unique
- `name` unique

7.5. role_qualifications

- id
- server_profile_id
- role_id
- granted_by_user_account_id
- granted_at
- revoked_at

Constraints:

- only one active qualification per server profile and role

7.6. availability_rules

- id
- server_profile_id
- day_of_week
- available_from_time
- available_until_time
- is_available
- created_at
- updated_at

Recurring availability rules define the normal weekly availability pattern used when generating the main schedule.

7.7. availability_overrides

- id
- server_profile_id
- starts_at
- ends_at
- is_available
- reason
- created_by_user_account_id
- created_at
- updated_at

Availability overrides define date-specific exceptions to the recurring availability rules. Examples include vacations, illness, school events, travel, or a one-time ability to serve a special liturgy.

7.8. service_templates

- id
- name
- description
- default_service_type
- is_active

7.9. service_template_positions

- id
- service_template_id
- role_id
- abbreviation
- required_count
- display_order

7.10. services

- id
- service_type
- title
- starts_at
- ends_at
- liturgical_color
- service_template_id
- created_by_user_account_id
- created_at
- updated_at
- cancelled_at

7.11. positions

- id
- service_id
- role_id
- abbreviation
- display_order
- required_count
- state

Recommended states:

- open
- requested
- assigned
- declined
- cancelled

7.12. assignments

- id

- service_id
- position_id
- server_profile_id
- assigned_by_user_account_id
- assigned_at
- declined_by_user_account_id
- declined_at
- cancelled_by_user_account_id
- cancelled_at
- state

7.13. volunteer_requests

- id
- service_id
- position_id
- server_profile_id
- submitted_by_user_account_id
- submitted_at
- reviewed_by_user_account_id
- reviewed_at
- state

Recommended states:

- pending
- accepted
- rejected
- cancelled

7.14. audit_events

- id
- actor_user_account_id
- actor_was_scheduler
- entity_type
- entity_id
- action
- summary
- occurred_at
- metadata_json

8. Authorization Rules

AUT-001. Only adult User Accounts shall authenticate.

AUT-002. A User Account may edit its own account contact details.

AUT-003. A User Account may edit Server Profiles that it manages.

AUT-004. A Managing Adult may submit Volunteer Requests for the Minor Server Profiles it manages.

AUT-005. A Managing Adult may decline Assignments for the Minor Server Profiles it manages.

AUT-006. A User Account may not submit a Volunteer Request for a Server Profile it does not manage, unless acting as a Scheduler.

AUT-007. A Scheduler may create Services, create Positions, assign Server Profiles, approve Volunteer Requests, and remove Assignments.

AUT-008. A Scheduler may add Role Qualifications to Minor Server Profiles.

AUT-009. Every write operation shall be authorized on the server, independent of any client-side UI restrictions.

AUT-010. The system shall use Auth.js for authentication.

AUT-011. The system shall support OAuth 2.0 or OpenID Connect authentication providers through Auth.js.

AUT-012. The system may support local credentials through Auth.js if local username and password accounts are required.

9. Scheduling Engine

The scheduling engine should be a pure TypeScript domain service where possible. It should accept inputs, produce a proposed rota, and avoid direct database writes until the Scheduler accepts the proposal.

Inputs:

- date range
- service templates or selected services
- active Server Profiles
- Role Qualifications
- recurring availability rules
- date-specific availability overrides
- existing Services
- existing Assignments
- existing Volunteer Requests

Outputs:

- proposed Assignments
- unfilled Positions
- warnings
- fairness notes, if available

SEN-001. The scheduling engine shall only propose Server Profiles that are qualified for the Position's Role unless the Scheduler explicitly allows exceptions.

SEN-002. The scheduling engine shall only propose Server Profiles that are available for the Service time.

SEN-003. The scheduling engine shall avoid assigning the same Server Profile more than once within the same Service.

SEN-004. The scheduling engine shall avoid assigning the same Server Profile to overlapping Services.

SEN-005. The scheduling engine should distribute Assignments fairly across qualified and available Server Profiles.

SEN-006. The scheduling engine shall report unfilled Positions.

SEN-007. The scheduling engine shall not persist generated Assignments until the Scheduler accepts the generated Rota.

SEN-008. The scheduling engine shall use recurring availability rules to create the main generated schedule.

SEN-009. The scheduling engine shall apply date-specific availability overrides after recurring availability rules.

10. Web Frontend

The frontend should be a practical scheduling interface rather than a marketing-style site.

Primary views:

- Calendar day view
- Calendar week view
- Calendar month view
- Service detail view
- Rota generation screen
- Server profile management
- Role qualification management
- Volunteer request review
- Account management
- Audit/change record view for Schedulers

WEB-001. The calendar shall be usable on desktop and mobile browsers.

WEB-002. Schedulers shall be able to create or edit Services from the calendar.

WEB-003. Adult users shall be able to view Assignments for themselves and managed Minor Server Profiles.

WEB-004. Adult users shall be able to submit Volunteer Requests for eligible managed Server Profiles.

WEB-005. Schedulers shall have a clear review interface for pending Volunteer Requests.

WEB-006. Schedulers shall be warned before replacing existing Services, Positions, Assignments, or Volunteer Requests during generated Rota acceptance.

11. API and Server Actions

The application may use Next.js server actions, route handlers, or a service-layer API. Regardless of transport, the domain operations should be explicit.

Recommended operations:

- createService

- updateService
- cancelService
- createPosition
- updatePosition
- submitVolunteerRequest
- reviewVolunteerRequest
- createAssignment
- declineAssignment
- removeAssignment
- generateRotaProposal
- acceptGeneratedRota
- addRoleQualification
- revokeRoleQualification
- writeAuditEvent

Each operation should:

1. validate input
2. authorize the actor
3. load required domain records
4. enforce domain rules
5. persist changes in a transaction
6. write audit events
7. return a typed result

12. Realtime Updates

The first implementation may refresh calendar data on a timer, as stated in the product requirements.

If timer-based refresh becomes inadequate, the system should add server-sent events or WebSockets for live rota updates.

Recommended starting point:

- poll visible calendar ranges every 30 to 60 seconds
- refresh immediately after local write operations
- add realtime subscriptions later if needed

13. Notifications

The first release should support email notifications. The notification design should remain open to additional notification channels later, such as SMS, push notifications, or parish-specific integrations.

Notifications should be implemented behind a provider interface so that the application can change providers without rewriting domain workflows.

Recommended initial channels:

- email for first release
- SMS as a later optional channel
- other notification providers as future extensions

Recommended notification events:

- Assignment created
- Assignment declined
- Volunteer Request submitted
- Volunteer Request accepted
- Volunteer Request rejected
- Rota published or updated

NOT-001. The first release shall support email notifications.

NOT-002. Notification delivery shall be implemented behind an internal notification provider interface.

NOT-003. The notification design shall allow additional channels, including SMS, without changing core rota workflows.

NOT-004. Notification events shall identify the affected Server Profile and the adult User Account receiving the notification.

NOT-005. Notifications for Minor Server Profiles shall be sent to the Managing Adult, not to the minor.

14. Migration From Existing Lua Prototype

The existing Lua files contain reusable concepts:

- `role_definitions.lua` maps to the `roles` table
- `mass_definitions.lua` maps to `service_templates` and `service_template_positions`
- `servers.lua` maps partly to `server_profiles`, `role_qualifications`, and availability
- `assignServersToMass` maps to the first TypeScript scheduling engine

The Lua code should not be used as the production runtime unless a later architectural decision reverses this document.

Migration steps:

1. Normalize role codes and names.
2. Convert Mass definitions into service templates.
3. Convert server records into Server Profiles.
4. Convert server role arrays into Role Qualifications.
5. Convert day-based availability into the first availability model.
6. Port the scheduling algorithm into TypeScript.
7. Add tests that compare known Lua-style inputs against expected TypeScript scheduler outputs.

15. Testing Requirements

TST-008. The scheduling engine shall have unit tests for qualification matching, availability matching, duplicate prevention, overlap prevention, and unfilled Position reporting.

TST-009. Authorization tests shall verify that User Accounts cannot manage unrelated Server Profiles.

TST-010. Authorization tests shall verify that Minor Server Profiles cannot authenticate.

TST-011. Authorization tests shall verify that only the Managing Adult can submit or decline on behalf of a Minor Server Profile.

TST-012. Scheduler workflow tests shall verify Service creation, Assignment creation, Volunteer Request approval, and generated Rota acceptance.

TST-013. Database migration tests should run in continuous integration.

16. Non-Functional Requirements

NFR-001. The system shall preserve data integrity through database constraints and transactional writes.

NFR-002. The system should render calendar views quickly for ordinary parish rota sizes.

NFR-003. The system should support multiple years of historical rota data.

NFR-004. The system shall keep application configuration outside source code.

NFR-005. The system shall support database backup and restore procedures.

NFR-006. The system shall produce useful server-side logs for failed authorization, failed validation, and scheduler actions.

17. Resolved Technical Decisions

Database access layer:: The system shall use Prisma for database access and migrations.

Authentication:: The system shall use Auth.js. OAuth 2.0 and OpenID Connect providers are supported through Auth.js. Local credentials may be added if required.

Notifications:: The first release shall support email notifications and shall keep the notification design open to additional providers and channels.

Availability:: The system shall use recurring weekly availability for normal schedule generation and date-specific overrides for exceptions.

Liturgical calendar data:: The first release shall not import liturgical calendar data from `calendarium-romanum`. The system should be designed so that liturgical calendar integration can be added later.

Deployment target:: The recommended first deployment target is a VPS using Docker Compose with `app` and `db` services. Docker Compose remains the local development baseline. Managed cloud deployment on AWS or Azure remains an optional future path.